Error Detection and Correction

Block Codes, Simple Parity Check, Checksum, CRC, LRC, Hamming Distance, Hamming Code.

Error control mechanism combines **error detection** and **error correction** to ensure reliable data transmission. It enables the receiver to detect frames that are lost or corrupted during transmission and notify the sender about these errors. The sender then retransmits the affected frames to ensure accurate delivery.

The two main strategies for handling transmission errors:

1. **Forward Error Correction (FEC):** The sender includes enough redundant data for the receiver to both detect and isolate the exact location of the error. The receiver fixes the error automatically without asking for a retransmission
2. **Backward Error Correction (Automatic Repeat Request - ARQ)**: The receiver explicitly requests the sender to resend the corrupted or missing data segments. This relies on positive acknowledgments (ACK), negative acknowledgments (NACK), and timeouts

Error-detecting codes : Error Detection and Retransmission Process involves detecting errors in transmitted data and retransmitting the affected data to ensure accurate and reliable communication. This method is usually more efficient when errors occur occasionally. Error Detection and Retransmission Process involves-

1. The sender adds redundant bits to the original data (dataword).
2. The encoded data (codeword) is transmitted.
3. The receiver checks the redundant bits.
4. If no error is found, the data is accepted.
5. If an error is detected, the receiver requests retransmission using **Automatic Repeat reQuest (ARQ)**.

Different error-detecting codes are

1. Simple Parity Check
2. Checksum
3. Longitudinal Redundancy Check (LRC)
4. Cyclic Redundancy Check (CRC)

Error-Correcting Codes : An error-correcting code (ECC) is a method of adding **extra bits (redundant information)** to data so that errors introduced during transmission or storage can be **detected and corrected** without needing the original data to be sent again.

Process involves-

1. The sender encodes the original data by adding extra check bits.
2. The encoded data is transmitted or stored.
3. If some bits change because of noise or hardware faults, the receiver uses the check bits to:
    - Detect that an error occurred.
    - Correct the error (if it is within the code's capability).

Different error-correcting codes:

1. Hamming codes.
2. Binary convolutional codes.
3. Reed-Solomon codes.
4. Low-Density Parity Check codes.

Block Codes

Block coding technique used to improve the reliability of data transmission by adding extra bits to the original data. In this method we divide our message into blocks, each of k bits, called datawords. We add r redundant bits to each block to make the length n = k + r. The resulting n-bit blocks are called codewords. I,e an n-bit unit containing data and check bits is referred to as an n-bit codeword

Example: Let us assume that k = 2 and n = 3. Table shows the list of datawords and codewords.

Datawords	*Codewords*
00	000
01	011
10	101
11	110

Simple Parity Check

A **parity bit** is an extra bit added to a group of data bits to detect errors during data transmission. The sender calculates the parity bit based on the number of **1s** in the data and appends it to the data before transmission. At the receiver, the parity is checked again. If the received parity does not match the expected parity, an error is detected.

Types of Parity

Even Parity: The total number of **1s** in the data (including the parity bit) is **even**. If the number of 1s in the original data is odd, the parity bit is set to **1**. If the number of 1s is already even, the parity bit is set to ***0.***

Data	Number of 1s	Parity Bit	Transmitted Data
1011001	4 (Even)	0	10110010
1011000	3 (Odd)	1	10110001

Odd Parity : The total number of **1s** in the data (including the parity bit) is **odd**. If the number of 1s is even in original data, the parity bit is set to **1**. If the number of 1s in original data is already odd, the parity bit is **0**.

Data	Number of 1s	Parity Bit	Transmitted Data
1011001	4 (Even)	1	10110011
1011000	3 (Odd)	0	10110000

Advantages

1. Simple and inexpensive to implement.
2. Detects all **single-bit errors**.
3. Widely used for basic error detection.

Limitations

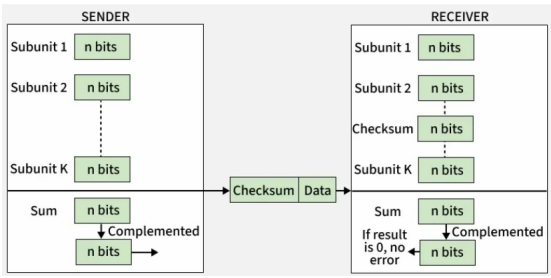
1. Cannot detect errors when an **even number of bits** are corrupted.
2. It can only **detect** errors; it **cannot correct** them.

Checksum

Checksum is an **error-detection technique** used in data communication to detect errors in transmitted data. It is commonly used in protocols such as IP, TCP, and UDP. Checksum works by summing all data segments using 1's complement arithmetic, taking the complement of the result, appending it to the data, and verifying at the receiver by checking if the final complemented sum (including checksum) equals zero.

The checksum method works as follows:

1. **Divide the data into fixed-size blocks** of bits, such as 8-bit or 16-bit words.
2. The **sender adds all the data blocks** using **one's complement addition**.
3. If there is a carry beyond the fixed number of bits, the carry is **wrapped around and added back** to the result.
4. The sender takes the **one's complement of the final sum**.
5. This value is called the **checksum**.
6. The sender sends the **original data along with the checksum** to the receiver.
7. The receiver adds all the received data blocks and the received checksum using the same one's complement addition.
8. The receiver takes/checks the final result:
    - **All 1s → No error detected**
    - **Anything other than all 1s → Error detected**



Example 1 - If the data unit to be transmitted is 10101001 00111001, the following procedure is used at Sender site and Receiver site

Sender Site:

```
10101001        subunit 1
00111001        subunit 2
11100010        sum (using 1s complement)
00011101        checksum (complement of sum)
```

Data transmitted to Receiver is:

10101001 00111001	00011101
Data	Checksum

Receiver Site:

```
10101001        subunit 1
00111001        subunit 2
00011101        checksum
11111111            sum
00000000    sum's complement
```

Result is zero, it means no error.

Example 2 - If the data transmitted along with checksum is 10101001 00111001 00011101. But the data received at destination is 00101001 10111001 00011101.

Receiver Site:

```
00101001          1st bit of subunit 1 is damaged
10111001           1st bit of subunit 2 is damaged
00011101          checksum
11111111             sum
00000000      Ok 1's complement
```

Although data is corrupted, the error is undetected. Because the changes in data 1 and data 2 offset each other, resulting in the same overall sum. This is a limitation of simple checksum – it cannot detect all types of errors, especially when multiple errors cancel each other out.

Longitudinal Redundancy Check (LRC)

It is an error-detection method. It is also known as two dimensional parity. In this method block of data bits are organized into rows and columns. Parity bit is calculated for each column and sent along with the data. The block of parity acts as the redundant bits. LRC can detect single-bit errors and many multiple-bit/burst errors, but some error patterns may go undetected.

The LRC method works as follows:

Sender side

1. The sender divides the data into rows/bytes.
2. It calculates the **LRC value column-wise** using parity.
3. The sender adds the LRC to the data.
4. The complete data + LRC is sent to the receiver.

Receiver side

1. The receiver receives the data and LRC.
2. It calculates the LRC again from the received data.
3. It compares the calculated LRC with the received LRC.
4. If they match → **no error is detected**.
5. If they don't match → **error is detected**.

Example:

Suppose we have: **Dataword: 10110010 1100101010011100,**

1. **It is divided into block**

   10110010
   11001010
   10011100

2. The LRC is calculated by checking the parity of each column using either even parity or odd parity. An additional row containing the parity bits is then added to the data.

   10110010
   11001010
   10011100

   11100100 -> Parity row (LRC)

3. Codeword = Dataword + LRC (parity) bits, which is sent by the sender to the receiver**:**

   10110010 1100101010011100 11100100

Example : If a block of 32 bits is to be transmitted, it is divided into matrix of four rows and eight columns which as shown in the following figure :

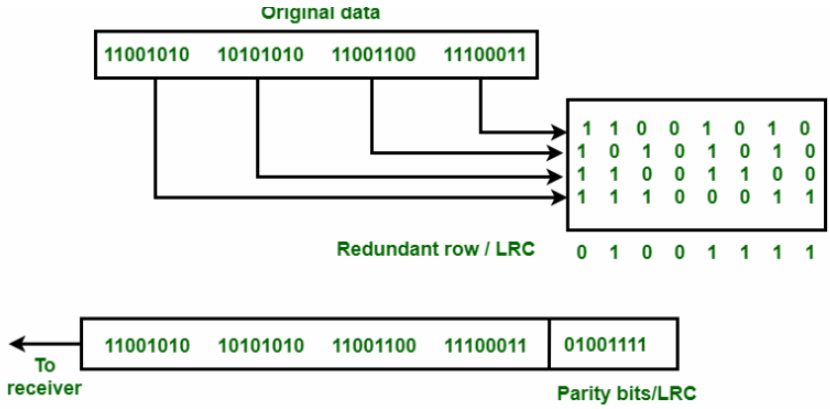


Figure: LRC

In this matrix of bits, a parity bit (odd or even) is calculated for each column. It means 32 bits data plus 8 redundant bits are transmitted to receiver. Whenever data reaches at the destination, receiver uses LRC to detect error in data. **Advantage:** LRC is used to detect burst errors. **Example:** Suppose 32 bit data plus LRC that was being transmitted is hit by a burst error of length 5 and some bits are corrupted as shown in the following figure :

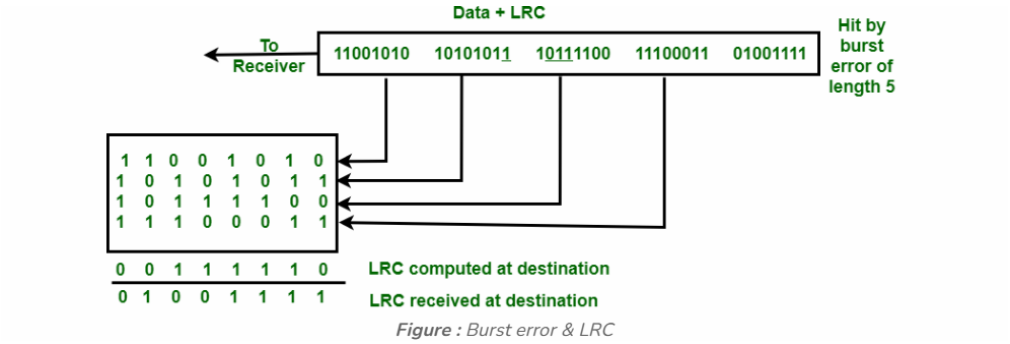


Figure : *Burst error & LRC*

The LRC received by the destination does not match with newly corrupted LRC. The destination comes to know that the data is erroneous, so it discards the data. **Disadvantage :** The main problem with LRC is that, it is not able to detect error if two bits in a data unit are damaged and two bits in exactly the same position in other data unit are also damaged. **Example :** If data 110011 010101 is changed to 010010110100.

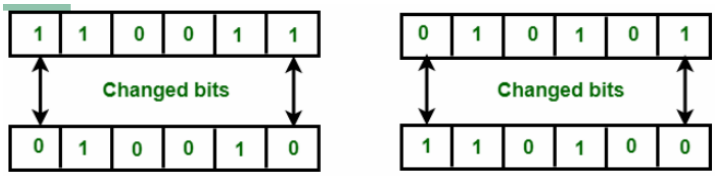


Figure : *Two bits at same bit position damaged in 2 data units*

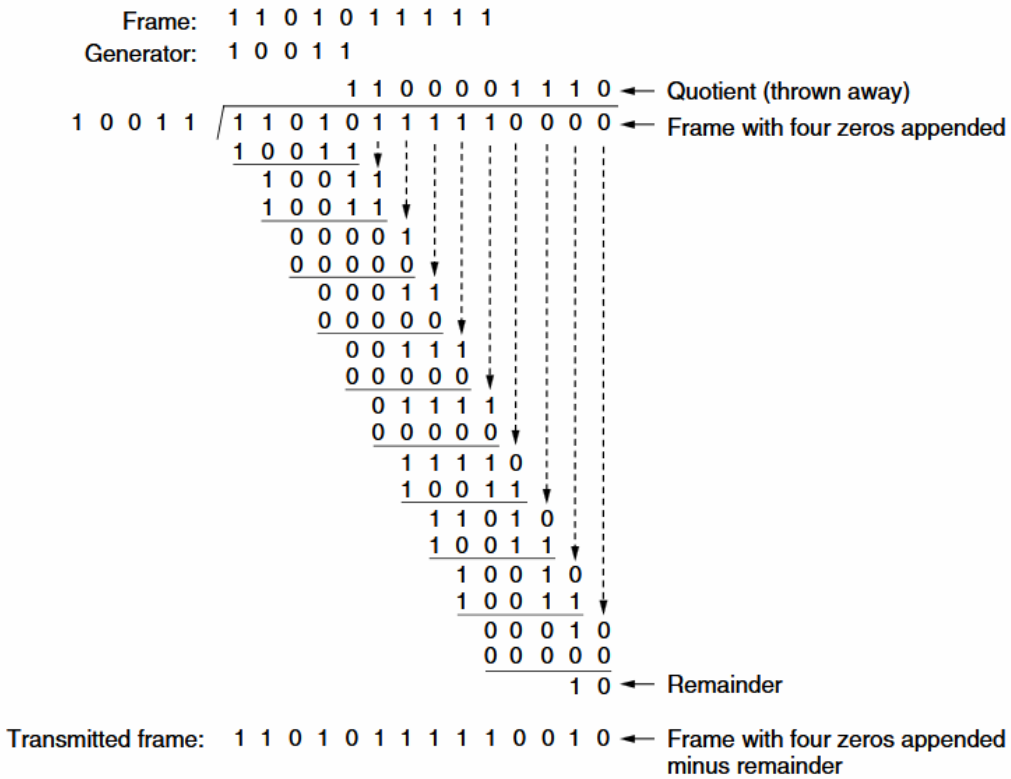
In this example 1st and 6th bit in one data unit is changed . Also the 1st and 6th bit in second unit is changed.

CRC (Cyclic Redundancy Check) is an error-detection technique used in data communication to detect errors in transmitted data. is also known as a **polynomial code.** CRC provides strong and reliable error detection, especially for burst errors. **Disadvantage:** It is more complex to implement than LRC.

The CRC method works as follows:

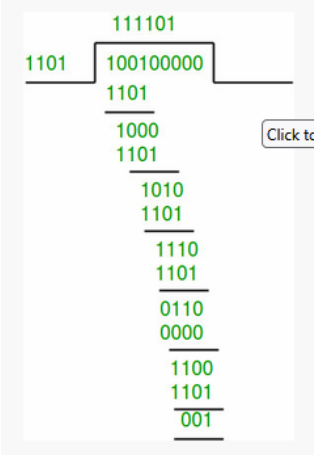
1. At the sender, the dataword is taken and a generator polynomial is selected.
2. The sender appends zeros to the dataword. The number of zeros is one less than the number of bits in the generator.
3. The resulting data is divided by the generator using modulo-2 (XOR) division.
4. The remainder obtained from the division is called the CRC. This CRC is added to the original dataword to form the codeword, which is transmitted.
5. At the receiver, the received codeword is divided by the same generator using XOR division. If the remainder is zero, no error is detected; if the remainder is non-zero, an error is detected.

Example: Figure illustrates the calculation for a frame 1101011111 using the generator G(x) = x 4 + x + 1.



Example 2: Data = 100100, Generator Polynomial (Key) = $x^3 + x^2 + 1$ (1101)

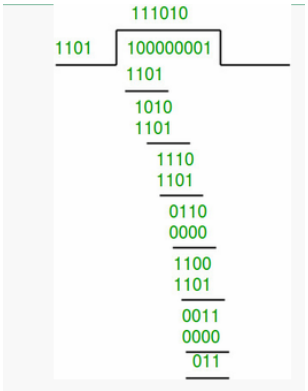
Sender Side



The remainder is 001. Thus the data sent is **100100001.**

Receiver Side

Let there be an error and code word received at the receiver side **100000001.**



As the remainder is not 0, hence there is some error detected in the receiver side.

Hamming Code

Hamming Code is an error-detecting and error-correcting technique used in digital communication and computer networks. It can **detect up to two-bit errors** and **correct one-bit errors** in a data unit. It was developed by **Richard Hamming**. During data transmission, bits may change because of noise or interference. Hamming Code adds extra parity bits to the original data so that the receiver can identify the position of an erroneous bit and correct it.

If there are **m data bits** and **r parity bits**, the number of parity bits required is determined by:

$$2^r \geq m + r + 1$$

Where: m = number of data bits and r = number of parity bits

Position of Parity Bits:

Parity bits are placed at positions that are powers of 2: **1, 2, 4, 8, 16, ...**
The remaining positions contain the data bits.
For example, for 4 data bits:

Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Bit	P1	P2	D1	P4	D2	D3	D4

Thus, a **(7,4) Hamming Code** contains 4 data bits and 3 parity bits.

Working Principle :

Each parity bit checks a particular group of bit positions.
For a 7-bit Hamming Code:

- **P1** checks positions **1, 3, 5, 7**
- **P2** checks positions **2, 3, 6, 7**
- **P4** checks positions **4, 5, 6, 7**

The parity can be **even parity** or **odd parity**. Even parity is commonly used.

Error Detection and Correction

At the receiver, parity checks are performed again.
The results form a **syndrome**. The syndrome gives the position of the erroneous bit.
For example:

$$S = S_4 S_2 S_1$$

If: $S_4 S_2 S_1 = 101_2 = 5$

then **bit 5 is incorrect**. The receiver corrects it by changing 0 → 1 or 1 → 0.If the syndrome is **000**, there is no detected single-bit error.

Example : Given the data bits 1011, use Hamming (7,4) code to find the parity bits and determine the final Hamming codeword.

Position	1	2	3	4	5	6	7
Bit	P1	P2	D1	P4	D2	D3	D4
Value	?	?	1	?	0	1	1

Using **even parity**:

- **P1 = 0**
- **P2 = 1**
- **P4 = 0**

Therefore, the final Hamming codeword is: 0110011

Example: A 7 bit hamming code is received as 1011011 assume even parity and state whether the received code is correct or wrong. if wrong locate the bit with error.

Solution: Received Hamming code: 1011011

Position	1	2	3	4	5	6	7
Bit	1	0	1	1	0	1	1

Parity bits are at positions 1, 2, and 4.

Step 1: Check P1

P1 checks positions 1, 3, 5, 7:
1+1+0+1=3
3 is odd → P1 check = 1 (error).

Step 2: Check P2

P2 checks positions 2, 3, 6, 7:
0+1+1+1=3
3 is odd → P2 check = 1 (error).

Step 3: Check P4

P4 checks positions 4, 5, 6, 7:
1+0+1+1=3

3 is odd → P4 check = 1 (error).

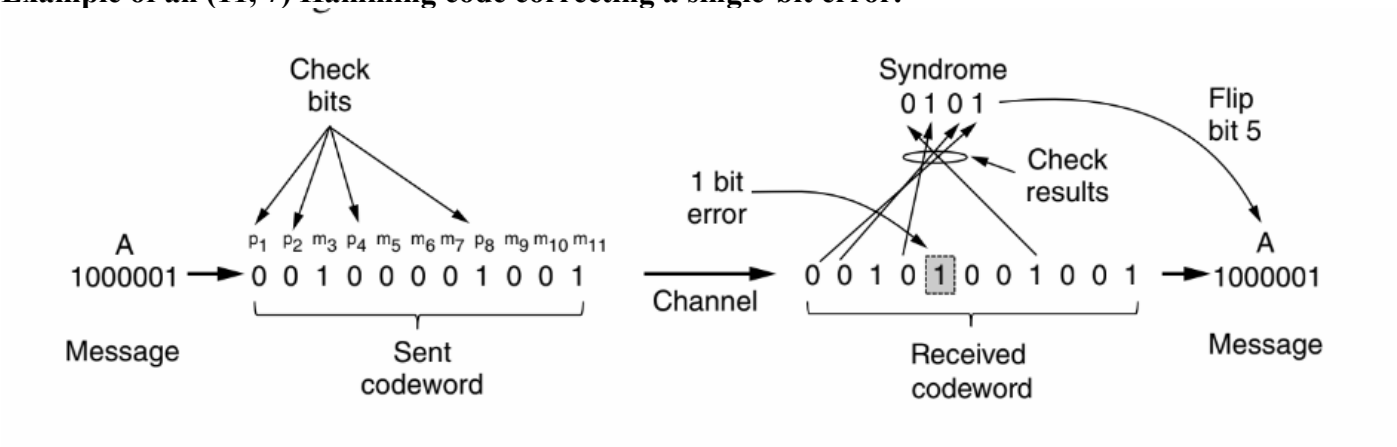
Step 4: Find the error position

The syndrome is: $S_4S_2S_1 = 111$

Convert binary to decimal: $111_2 = 7$

Therefore, bit position 7 is erroneous.

Example of an (11, 7) Hamming code correcting a single-bit error.



.